

PROJECT INTERIM REPORT

ON

DIGITAL STOPWATCH LOGIC SIMULATION

(Using Core Java & OOP Concepts)

Course:

RU-100-01-00012: Object Oriented Programming with Java
SESSION: 2025-26



SUBMITTED BY:
Yash Kumar Singh
ERP: RU 25 11630

Bachelor of Technology in Computer Science & Engineering
in association with
GOOGLE

UNDER THE GUIDANCE OF:
Mr. Ashish Shivhare

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
RUNGTA INTERNATIONAL SKILLS UNIVERSITY
BHILAI, CG

(April 2026)

Abstract

A digital stopwatch is a fundamental timekeeping tool used to measure elapsed time accurately. This project focuses on designing and implementing a **Digital Stopwatch Logic Simulation** using **Object-Oriented Programming (OOP) principles** in Core Java. The stopwatch functionality is encapsulated inside a dedicated **Stopwatch** class, where timing is tracked using **System.currentTimeMillis()** for reliable time measurement. The system supports key operations such as **start**, **stop (pause)**, **reset**, and **getElapsedTime**. To ensure correct behavior, the stopwatch maintains internal state variables including **startTime**, **elapsedTime**, and **isRunning**. When the stopwatch is started and stopped multiple times, the elapsed time is accumulated properly, demonstrating real-world state management. This project highlights important OOP concepts such as **encapsulation**, **class-based design**, **method interaction**, and **state transitions**. It provides a clean and reusable console-based simulation suitable for academic evaluation and learning.

Introduction

A stopwatch measures time intervals by tracking the duration between a start event and a stop event. In real-life applications, stopwatch behavior often requires pausing and resuming while still keeping total elapsed time accurate. Similarly, a software stopwatch must maintain internal state safely to prevent incorrect calculations.

In this project, we design and implement a Digital Stopwatch Logic Simulation using Core Java and OOP concepts. The stopwatch logic is controlled through a dedicated class that interacts through methods like **start()**, **stop()**, **reset()**, and **getElapsedTime()**. The use of encapsulation ensures that timing variables remain private and only update through well-defined methods.

This project serves as a practical example of time-based computation and state control in Java, helping students understand how objects behave over time.

Objectives

- Design a digital stopwatch simulation using **Core Java** and **OOP concepts**.

- Encapsulate timing logic inside a dedicated `Stopwatch` class.
 - Implement accurate elapsed time tracking using `System.currentTimeMillis()`.
 - Support operations:
 - **Start**
 - **Stop (Pause)**
 - **Reset**
 - **Get elapsed time**
 - Demonstrate proper **state management** using internal variables and flags.
-

Scope

- This project focuses on a console-based stopwatch simulation.
 - The stopwatch tracks elapsed time accurately across multiple start/stop cycles.
 - The current scope includes:
 - Single stopwatch instance
 - Core timing operations (start, stop, reset, retrieve elapsed time)
 - In future, it can be extended with:
 - Better UI (GUI using Java Swing/JavaFX)
 - Saving stopwatch history
 - Multiple stopwatch instances or laps
-

System Requirements

Hardware Requirements

- Processor: Intel Core i3 or equivalent (minimum)
- RAM: 4 GB (minimum)
- Storage: 500 MB free disk space

Software Requirements

- Operating System: Windows 10/11, Linux, or macOS
- Programming Language: Java (JDK 11 or above)
- IDE: VS Code / Eclipse / IntelliJ IDEA / EditPlus
- Java Runtime Environment (JRE)

Methodology

The stopwatch simulation follows a clear step-by-step process:

1. **Initialize** the `Stopwatch` object.
 2. **Start** the stopwatch:
 - Store current time in `startTime`
 - Set `isRunning = true`
 3. **Stop (Pause)** the stopwatch:
 - Calculate elapsed time from `currentTime - startTime`
 - Add it to `elapsedTime`
 - Set `isRunning = false`
 4. **Resume** (Start again):
 - Repeat the same start/stop logic
 - Elapsed time accumulates correctly
 5. **Reset** the stopwatch:
 - Set `startTime = 0, elapsedTime = 0, isRunning = false`
 6. **Get elapsed time**:
 - If running: return `(currentTime - startTime) + elapsedTime`
 - If not running: return stored `elapsedTime`
-

Flowchart (Logical Flow)

Start

→ Create Stopwatch Object

→ User Selects Operation

→ `start()`?

- Yes → Store start time, set running → Continue
- No → Check `stop()`?
 - `stop()`?
- Yes → Calculate & accumulate elapsed time → Stop running → Continue
- No → Check `reset()`?
 - `reset()`?
- Yes → Clear all values → Continue

- No → Check `getElapsedTime()`?
 - `getElapsedTime()` → Display time
 - Repeat until Exit
 - End
-

Class Diagram (Text Representation)

Stopwatch Class

Private Variables

- `long startTime`
- `long elapsedTime`
- `boolean isRunning`

Public Methods

- `void start()`
- `void stop()`
- `void reset()`
- `long getElapsedTime()`

Relationships

- A single object of `Stopwatch` manages the entire stopwatch behavior.
-

Implementation (Logic Explanation)

The project is mainly implemented using a single class:

1) `Stopwatch` Class

- **Encapsulation:**
`startTime`, `elapsedTime`, and `isRunning` are declared as `private` so they cannot be modified externally.

- **start() method:**
Sets `startTime` only when the stopwatch is not already running and changes the running status.
- **stop() method:**
Computes elapsed duration since `startTime`, adds it to `elapsedTime`, and stops further counting.
- **reset() method:**
Clears all internal timing variables and stops the stopwatch.
- **getElapsedTime() method:**
Returns:
 - Accumulated elapsed time when stopped
 - Updated elapsed time when running

Main/Test Logic

A `main` method can be used to demonstrate the stopwatch actions in sequence: Start → Wait → Stop → Check time → Resume → Check time → Reset → Check time.

Sample Input and Output (Reference)

Example Console Output:

Stopwatch Started...
Stopwatch Stopped...
Elapsed Time: 2500 ms

Resuming Stopwatch...
Elapsed Time: 4000 ms

Stopwatch Reset.
Elapsed Time: 0 ms

Future Enhancements

- Add **lap functionality** (store split times).
- Create a **GUI** using Swing/JavaFX.
- Allow **multiple stopwatch instances**.
- Maintain **history log** of stopwatch sessions (store results in file/database).

- Provide a countdown mode along with stopwatch mode.
-

Gantt Chart

A Gantt chart can be structured like this:

- **Week 1:** Planning & requirement analysis
 - **Week 2:** Class design & flowchart/class diagram
 - **Week 3:** Coding `Stopwatch` logic
 - **Week 4:** Testing with multiple start/stop/resume cases
 - **Week 5:** Documentation & report finalization
-

Conclusion

The Digital Stopwatch Logic Simulation project successfully demonstrates Core Java programming and key Object-Oriented Programming concepts. It achieves correct time tracking by using `System.currentTimeMillis()` and properly managing internal state through encapsulation and method-driven logic. The project supports start, stop (pause), resume, reset, and elapsed time retrieval while accumulating time accurately across multiple cycles.

Overall, this simulation provides a strong learning experience in state management, debugging, and OOP-based software design, with scope for future enhancements like GUI and lap tracking.

References

- [1] H. Schildt, *Java: The Complete Reference*, 11th ed. New York: McGraw-Hill Education, 2018.
 - [2] B. Eckel, *Thinking in Java*, 4th ed. Upper Saddle River, NJ: Prentice Hall, 2006.
 - [3] Oracle, "Java Documentation – The Java Tutorials," Oracle. [Online]. Available: <https://docs.oracle.com/javase/tutorial/>
 - [4] GeeksforGeeks, "Object Oriented Programming in Java," GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/object-oriented-programming-oops-concept-in-java/>
-